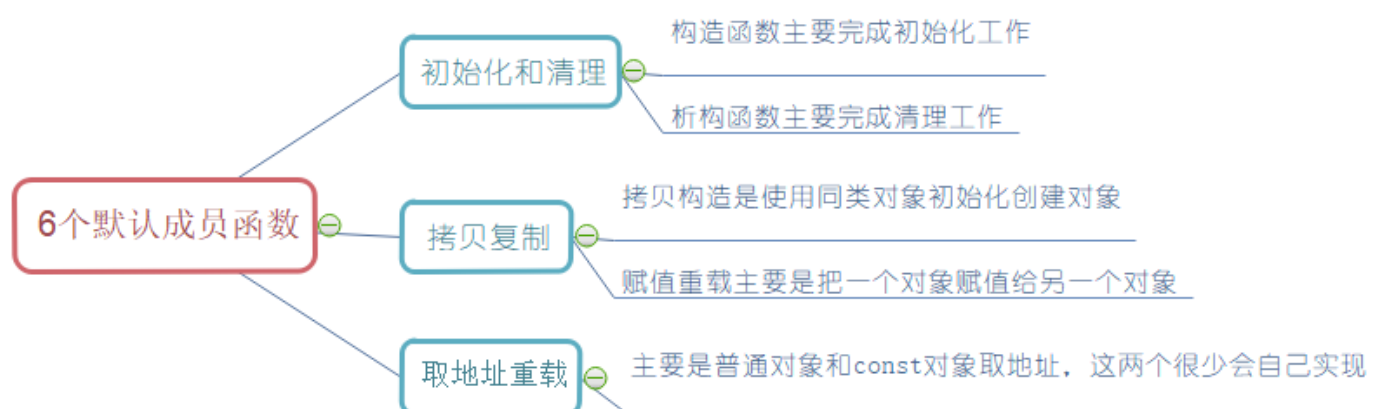


3.类和对象(中)

1. 类的默认成员函数

默认成员函数就是用户没有显式实现，编译器会自动生成的成员函数称为默认成员函数。一个类，我们不写的情况下编译器会默认生成以下6个默认成员函数，需要注意的是这6个中最重要的是前4个，最后两个取地址重载不重要，我们稍微了解一下即可。其次就是C++11以后还会增加两个默认成员函数，移动构造和移动赋值，这个我们后面再讲解。默认成员函数很重要，也比较复杂，我们要从两个方面去学习：

- 第一：我们不写时，编译器默认生成的函数行为是什么，是否满足我们的需求。
- 第二：编译器默认生成的函数不满足我们的需求，我们需要自己实现，那么如何自己实现？



2. 构造函数

构造函数是特殊的成员函数，需要注意的是，构造函数虽然名称叫构造，但是构造函数的主要任务并不是开空间创建对象(我们常使用的局部对象是栈帧创建时，空间就开好了)，而是对象实例化时初始化对象。构造函数的本质是要替代我们以前Stack和Date类中写的Init函数的功能，构造函数自动调用的特点就完美的替代的了Init。

构造函数的特点：

1. 函数名与类名相同。
2. 无返回值。(返回值啥都不需要给，也不需要写void，不要纠结，C++规定如此)
3. 对象实例化时系统会自动调用对应的构造函数。
4. 构造函数可以重载。
5. 如果类中没有显式定义构造函数，则C++编译器会自动生成一个无参的默认构造函数，一旦用户显式定义编译器将不再生成。

6. 无参构造函数、全缺省构造函数、我们不写构造时编译器默认生成的构造函数，都叫做默认构造函数。但是这三个函数有且只有一个存在，不能同时存在。无参构造函数和全缺省构造函数虽然构成函数重载，但是调用时会存在歧义。要注意很多同学会认为默认构造函数是编译器默认生成那个叫默认构造，实际上无参构造函数、全缺省构造函数也是默认构造，总结一下就是不传实参就可以调用的构造就叫默认构造。
7. 我们不写，编译器默认生成的构造，对内置类型成员变量的初始化没有要求，也就是说是否初始化是不确定的，看编译器。对于自定义类型成员变量，要求调用这个成员变量的默认构造函数初始化。如果这个成员变量，没有默认构造函数，那么就会报错，我们要初始化这个成员变量，需要用初始化列表才能解决，初始化列表，我们下个章节再细细讲解。

说明：C++把类型分成内置类型(基本类型)和自定义类型。内置类型就是语言提供的原生数据类型，如：int/char/double/指针等，自定义类型就是我们使用class/struct等关键字自己定义的类型。

```
1  #include<iostream>
2  using namespace std;
3  class Date
4  {
5  public:
6      // 1.无参构造函数
7      Date()
8      {
9          _year = 1;
10         _month = 1;
11         _day = 1;
12     }
13
14     // 2.带参构造函数
15     Date(int year, int month, int day)
16     {
17         _year = year;
18         _month = month;
19         _day = day;
20     }
21
22     // 3.全缺省构造函数
23     /*Date(int year = 1, int month = 1, int day = 1)
24     {
25         _year = year;
26         _month = month;
27         _day = day;
28     }*/
29
30     void Print()
31     {
32         cout << _year << "/" << _month << "/" << _day << endl;
```

```

33     }
34 private:
35     int _year;
36     int _month;
37     int _day;
38 };
39
40 int main()
41 {
42     // 如果留下三个构造中的第二个带参构造，第一个和第三个注释掉
43     // 编译报错: error C2512: "Date": 没有合适的默认构造函数可用
44     Date d1; // 调用默认构造函数
45     Date d2(2025, 1, 1); // 调用带参的构造函数
46
47     // 注意: 如果通过无参构造函数创建对象时，对象后面不用跟括号，否则编译器无法
48     // 区分这里是函数声明还是实例化对象
49     // warning C4930: "Date d3(void)": 未调用原型函数(是否是有意用变量定义的?)
50     Date d3();
51
52     d1.Print();
53     d2.Print();
54
55     return 0;
56 }

```

```

1  #include<iostream>
2  using namespace std;
3
4  typedef int STDataType;
5  class Stack
6  {
7  public:
8      Stack(int n = 4)
9      {
10         _a = (STDataType*)malloc(sizeof(STDataType) * n);
11         if (nullptr == _a)
12         {
13             perror("malloc申请空间失败");
14             return;
15         }
16
17         _capacity = n;
18         _top = 0;
19     }
20     // ...

```

```

21 private:
22     STDataType* _a;
23     size_t _capacity;
24     size_t _top;
25 };
26
27 // 两个Stack实现队列
28 class MyQueue
29 {
30 public:
31     //编译器默认生成MyQueue的构造函数调用了Stack的构造，完成了两个成员的初始化
32 private:
33     Stack pushst;
34     Stack popst;
35 };
36
37 int main()
38 {
39     MyQueue mq;
40
41     return 0;
42 }

```

3. 析构函数

析构函数与构造函数功能相反，析构函数不是完成对对象本身的销毁，比如局部对象是存在栈帧的，函数结束栈帧销毁，他就释放了，不需要我们管，C++规定对象在销毁时会自动调用析构函数，完成对象中资源的清理释放工作。析构函数的功能类比我们之前Stack实现的Destroy功能，而像Date没有Destroy，其实就是没有资源需要释放，所以严格说Date是不需要析构函数的。

析构函数的特点：

1. 析构函数名是在类名前加上字符 ~。
2. 无参数无返回值。（这里跟构造类似，也不需要加void）
3. 一个类只能有一个析构函数。若未显式定义，系统会自动生成默认的析构函数。
4. 对象生命周期结束时，系统会自动调用析构函数。
5. 跟构造函数类似，我们不写编译器自动生成的析构函数对内置类型成员不做处理，自定类型成员会调用他的析构函数。
6. 还需要注意的是我们显示写析构函数，对于自定义类型成员也会调用他的析构，也就是说自定义类型成员无论什么情况都会自动调用析构函数。

7. 如果类中没有申请资源时，析构函数可以不写，直接使用编译器生成的默认析构函数，如Date；如果默认生成的析构就可以用，也就不需要显示写析构，如MyQueue；但是有资源申请时，一定要自己写析构，否则会造成资源泄漏，如Stack。
8. 一个局部域的多个对象，C++规定后定义的先析构。

```
1  #include<iostream>
2  using namespace std;
3
4  typedef int STDataType;
5  class Stack
6  {
7  public:
8      Stack(int n = 4)
9      {
10         _a = (STDataType*)malloc(sizeof(STDataType) * n);
11         if (nullptr == _a)
12         {
13             perror("malloc申请空间失败");
14             return;
15         }
16
17         _capacity = n;
18         _top = 0;
19     }
20
21     ~Stack()
22     {
23         cout << "~Stack()" << endl;
24
25         free(_a);
26         _a = nullptr;
27         _top = _capacity = 0;
28     }
29
30 private:
31     STDataType* _a;
32     size_t _capacity;
33     size_t _top;
34 };
35
36 // 两个Stack实现队列
37 class MyQueue
38 {
39 public:
40     //编译器默认生成MyQueue的析构函数调用了Stack的析构，释放的Stack内部的资源
```

```

41
42     // 显示写析构，也会自动调用Stack的析构
43     /*~MyQueue()
44     {}*/
45 private:
46     Stack pushst;
47     Stack popst;
48 };
49
50 int main()
51 {
52     Stack st;
53
54     MyQueue mq;
55
56     return 0;
57 }

```

对比一下用C++和C实现的Stack解决之前括号匹配问题isValid，我们发现有了构造函数和析构函数确实方便了很多，不会再忘记调用Init和Destory函数了，也方便了不少。

```

1  #include<iostream>
2  using namespace std;
3
4  // 用最新加了构造和析构的C++版本Stack实现
5  bool isValid(const char* s) {
6      Stack st;
7
8      while (*s)
9      {
10         if (*s == '[' || *s == '(' || *s == '{')
11         {
12             st.Push(*s);
13         }
14         else
15         {
16             // 右括号比左括号多，数量匹配问题
17             if (st.Empty())
18             {
19                 return false;
20             }
21
22             // 栈里面取左括号
23             char top = st.Top();
24             st.Pop();

```

```

25
26         // 顺序不匹配
27         if ((*s == ']' && top != '[')
28             || (*s == '}' && top != '{')
29             || (*s == ')' && top != '('))
30         {
31             return false;
32         }
33     }
34
35     ++s;
36 }
37
38 // 栈为空, 返回真, 说明数量都匹配 左括号多, 右括号少匹配问题
39 return st.Empty();
40 }
41
42 // 用之前C版本Stack实现
43 bool isValid(const char* s) {
44     ST st;
45     STInit(&st);
46     while (*s)
47     {
48         // 左括号入栈
49         if (*s == '(' || *s == '[' || *s == '{')
50         {
51             STPush(&st, *s);
52         }
53         else // 右括号取栈顶左括号尝试匹配
54         {
55             if (STEmpty(&st))
56             {
57                 STDestroy(&st);
58                 return false;
59             }
60
61             char top = STTop(&st);
62             STPop(&st);
63
64             // 不匹配
65             if ((top == '(' && *s != ')')
66                 || (top == '{' && *s != '}')
67                 || (top == '[' && *s != ']'))
68             {
69                 STDestroy(&st);
70                 return false;
71             }

```

```

72         }
73
74         ++s;
75     }
76
77     // 栈不为空, 说明左括号比右括号多, 数量不匹配
78     bool ret = STEmpty(&st);
79     STDestroy(&st);
80
81     return ret;
82 }
83
84 int main()
85 {
86     cout << isValid("[()][ ]") << endl;
87     cout << isValid("[ ][ ]") << endl;
88
89
90     return 0;
91 }

```

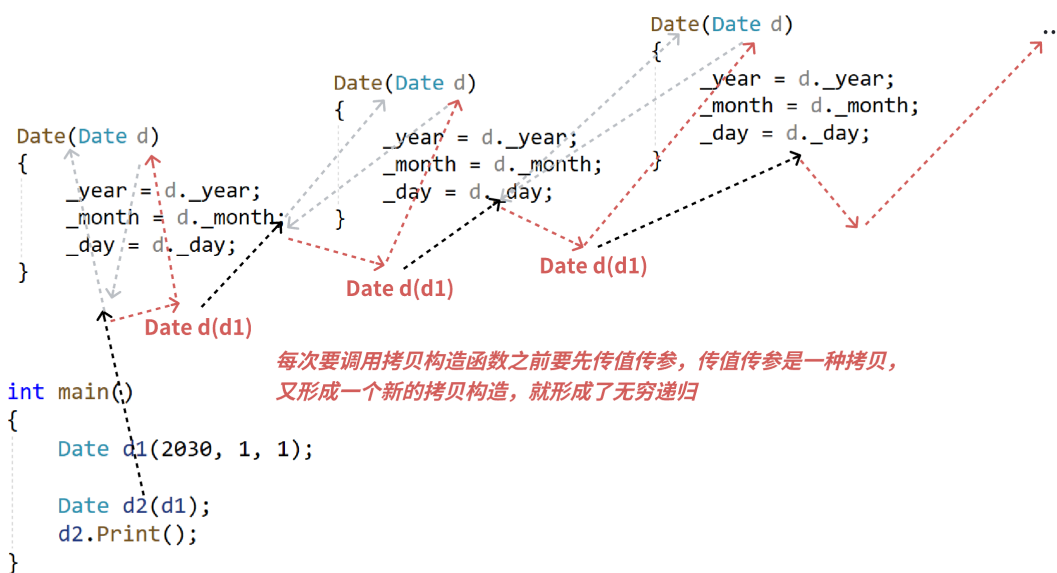
4. 拷贝构造函数

如果一个构造函数的第一个参数是自身类类型的引用，且任何额外的参数都有默认值，则此构造函数也叫做拷贝构造函数，也就是说拷贝构造是一个特殊的构造函数。

拷贝构造的特点：

1. 拷贝构造函数是构造函数的一个重载。
2. 拷贝构造函数的第一个参数必须是类类型对象的引用，使用传值方式编译器直接报错，因为语法逻辑上会引发无穷递归调用。拷贝构造函数也可以多个参数，但是第一个参数必须是类类型对象的引用，后面的参数必须有缺省值。
3. C++规定自定义类型对象进行拷贝行为必须调用拷贝构造，所以这里自定义类型传值传参和传值返回都会调用拷贝构造完成。
4. 若未显式定义拷贝构造，编译器会生成自动生成拷贝构造函数。自动生成的拷贝构造对内置类型成员变量会完成值拷贝/浅拷贝(一个字节一个字节的拷贝)，对自定义类型成员变量会调用他的拷贝构造。

5. 像Date这样的类成员变量全是内置类型且没有指向什么资源，编译器自动生成的拷贝构造就可以完成需要的拷贝，所以不需要我们显示实现拷贝构造。像Stack这样的类，虽然也都是内置类型，但是_a指向了资源，编译器自动生成的拷贝构造完成的值拷贝/浅拷贝不符合我们的需求，所以需要我们自己实现深拷贝(对指向的资源也进行拷贝)。像MyQueue这样的类型内部主要是自定义类型Stack成员，编译器自动生成的拷贝构造会调用Stack的拷贝构造，也不需要我们显示实现MyQueue的拷贝构造。这里还有一个小技巧，如果一个类显示实现了析构并释放资源，那么他就需要显示写拷贝构造，否则就不需要。
6. 传值返回会产生一个临时对象调用拷贝构造，传值引用返回，返回的是返回对象的别名(引用)，没有产生拷贝。但是如果返回对象是一个当前函数局部域的局部对象，函数结束就销毁了，那么使用引用返回是有问题的，这时的引用相当于一个野引用，类似一个野指针一样。传引用返回可以减少拷贝，但是一定要确保返回对象，在当前函数结束后还在，才能用引用返回。



```
1 #include<iostream>
2 using namespace std;
3
4 class Date
5 {
6 public:
7     Date(int year = 1, int month = 1, int day = 1)
8     {
9         _year = year;
10        _month = month;
11        _day = day;
12    }
13
14    // 编译报错: error C2652: "Date": 非法的复制构造函数: 第一个参数不应是"Date"
15    //Date(Date d)
16    Date(const Date& d)
```

```

17     {
18         _year = d._year;
19         _month = d._month;
20         _day = d._day;
21     }
22
23     Date(Date* d)
24     {
25         _year = d->_year;
26         _month = d->_month;
27         _day = d->_day;
28     }
29
30     void Print()
31     {
32         cout << _year << "-" << _month << "-" << _day << endl;
33     }
34 private:
35     int _year;
36     int _month;
37     int _day;
38 };
39
40 void Func1(Date d)
41 {
42     cout << &d << endl;
43     d.Print();
44 }
45
46 // Date Func2()
47 Date& Func2()
48 {
49     Date tmp(2024, 7, 5);
50     tmp.Print();
51
52     return tmp;
53 }
54
55 int main()
56 {
57     Date d1(2024, 7, 5);
58     // C++规定自定义类型对象进行拷贝行为必须调用拷贝构造，所以这里传值传参要调用拷贝
构造
59     // 所以这里的d1传值传参给d要调用拷贝构造完成拷贝，传引用传参可以较少这里的拷贝
60     Func1(d1);
61
62     cout << &d1 << endl;

```

```

63
64     // 这里可以完成拷贝，但是不是拷贝构造，只是一个普通的构造
65     Date d2(&d1);
66     d1.Print();
67     d2.Print();
68
69     //这样写才是拷贝构造，通过同类型的对象初始化构造，而不是指针
70     Date d3(d1);
71     d2.Print();
72
73     // 也可以这样写，这里也是拷贝构造
74     Date d4 = d1;
75     d2.Print();
76
77     // Func2返回了一个局部对象tmp的引用作为返回值
78     // Func2函数结束，tmp对象就销毁了，相当于了一个野引用
79     Date ret = Func2();
80     ret.Print();
81
82     return 0;
83 }

```

```

1  #include<iostream>
2  using namespace std;
3
4  typedef int STDataType;
5  class Stack
6  {
7  public:
8      Stack(int n = 4)
9      {
10         _a = (STDataType*)malloc(sizeof(STDataType) * n);
11         if (nullptr == _a)
12         {
13             perror("malloc申请空间失败");
14             return;
15         }
16
17         _capacity = n;
18         _top = 0;
19     }
20
21     Stack(const Stack& st)
22     {
23         // 需要对_a指向资源创建同样大的资源再拷贝值

```

```

24         _a = (STDataType*)malloc(sizeof(STDataType) * st._capacity);
25         if (nullptr == _a)
26         {
27             perror("malloc申请空间失败!!!");
28             return;
29         }
30
31         memcpy(_a, st._a, sizeof(STDataType) * st._top);
32
33         _top = st._top;
34         _capacity = st._capacity;
35     }
36
37     void Push(STDataType x)
38     {
39         if (_top == _capacity)
40         {
41             int newcapacity = _capacity * 2;
42             STDataType* tmp = (STDataType*)realloc(_a, newcapacity *
sizeof(STDataType));
43             if (tmp == NULL)
44             {
45                 perror("realloc fail");
46                 return;
47             }
48
49             _a = tmp;
50             _capacity = newcapacity;
51         }
52
53         _a[_top++] = x;
54     }
55
56
57     ~Stack()
58     {
59         cout << "~Stack()" << endl;
60
61         free(_a);
62         _a = nullptr;
63         _top = _capacity = 0;
64     }
65
66 private:
67     STDataType* _a;
68     size_t _capacity;
69     size_t _top;

```

```

70 };
71
72 // 两个Stack实现队列
73 class MyQueue
74 {
75 public:
76 private:
77     Stack pushst;
78     Stack popst;
79 };
80
81 int main()
82 {
83     Stack st1;
84     st1.Push(1);
85     st1.Push(2);
86     // Stack不显示实现拷贝构造，用自动生成的拷贝构造完成浅拷贝
87     // 会导致st1和st2里面的_a指针指向同一块资源，析构时会析构两次，程序崩溃
88     Stack st2 = st1;
89
90     MyQueue mq1;
91     // MyQueue自动生成的拷贝构造，会自动调用Stack拷贝构造完成pushst/popst
92     // 的拷贝，只要Stack拷贝构造自己实现了深拷贝，他就没有问题
93     MyQueue mq2 = mq1;
94
95     return 0;
96 }

```

5. 赋值运算符重载

5.1 运算符重载

- 当运算符被用于类类型的对象时，C++语言允许我们通过运算符重载的形式指定新的含义。C++规定类类型对象使用运算符时，必须转换成调用对应运算符重载，若没有对应的运算符重载，则会编译报错。
- 运算符重载是具有特殊名字的函数，他的名字是由operator和后面要定义的运算符共同构成。和其他函数一样，它也具有其返回类型和参数列表以及函数体。
- 重载运算符函数的参数个数和该运算符作用的运算对象数量一样多。一元运算符有一个参数，二元运算符有两个参数，二元运算符的左侧运算对象传给第一个参数，右侧运算对象传给第二个参数。
- 如果一个重载运算符函数是成员函数，则它的第一个运算对象默认传给隐式的this指针，因此运算符重载作为成员函数时，参数比运算对象少一个。
- 运算符重载以后，其优先级和结合性与对应的内置类型运算符保持一致。

- 不能通过连接语法中没有的符号来创建新的操作符：比如operator@。
- `.* :: sizeof ?: .` 注意以上5个运算符不能重载。(选择题里面常考，大家要记一下)
- 重载操作符至少有一个类类型参数，不能通过运算符重载改变内置类型对象的含义，如：`int`
`operator+(int x, int y)`
- 一个类需要重载哪些运算符，是看哪些运算符重载后有意义，比如Date类重载operator-就有意义，但是重载operator+就没有意义。
- 重载++运算符时，有前置++和后置++，运算符重载函数名都是operator++，无法很好的区分。C++规定，后置++重载时，增加一个int形参，跟前置++构成函数重载，方便区分。
- 重载<<和>>时，需要重载为全局函数，因为重载为成员函数，this指针默认抢占了第一个形参位置，第一个形参位置是左侧运算对象，调用时就变成了 对象<<cout，不符合使用习惯和可读性。重载为全局函数把ostream/istream放到第一个形参位置就可以了，第二个形参位置当类类型对象。

```

1 #include<iostream>
2 using namespace std;
3
4 // 编译报错：“operator +”必须至少有一个类类型的形参
5 int operator+(int x, int y)
6 {
7     return x - y;
8 }
9
10 class A
11 {
12 public:
13     void func()
14     {
15         cout << "A::func()" << endl;
16     }
17 };
18
19 typedef void(A::*PF)(); //成员函数指针类型
20
21 int main()
22 {
23     // C++规定成员函数要加&才能取到函数指针
24     PF pf = &A::func;
25
26     A obj; //定义ob类对象temp
27
28     // 对象调用成员函数指针时，使用.*运算符
29     (obj.*pf)();

```

```
30
31     return 0;
32 }
```

```
1  #include<iostream>
2  using namespace std;
3
4  class Date
5  {
6  public:
7      Date(int year = 1, int month = 1, int day = 1)
8      {
9          _year = year;
10         _month = month;
11         _day = day;
12     }
13
14     void Print()
15     {
16         cout << _year << "-" << _month << "-" << _day << endl;
17     }
18
19     //private:
20     int _year;
21     int _month;
22     int _day;
23 };
24
25 // 重载为全局的面临对象访问私有成员变量的问题
26 // 有几种方法可以解决:
27 // 1、成员放公有
28 // 2、Date提供getxxx函数
29 // 3、友元函数
30 // 4、重载为成员函数
31 bool operator==(const Date& d1, const Date& d2)
32 {
33     return d1._year == d2._year
34         && d1._month == d2._month
35         && d1._day == d2._day;
36 }
37
38 int main()
39 {
40     Date d1(2024, 7, 5);
41     Date d2(2024, 7, 6);
```

```

42
43 // 运算符重载函数可以显示调用
44 operator==(d1, d2);
45
46 // 编译器会转换成 operator==(d1, d2);
47 d1 == d2;
48
49 return 0;
50 }

```

```

1 #include<iostream>
2 using namespace std;
3
4 class Date
5 {
6 public:
7     Date(int year = 1, int month = 1, int day = 1)
8     {
9         _year = year;
10        _month = month;
11        _day = day;
12    }
13
14    void Print()
15    {
16        cout << _year << "-" << _month << "-" << _day << endl;
17    }
18
19    bool operator==(const Date& d)
20    {
21        return _year == d._year
22            && _month == d._month
23            && _day == d._day;
24    }
25
26    Date& operator++()
27    {
28        cout << "前置++" << endl;
29        //...
30        return *this;
31    }
32
33    Date operator++(int)
34    {
35        Date tmp;

```

```

36         cout << "后置++" << endl;
37
38         //...
39         return tmp;
40     }
41 private:
42     int _year;
43     int _month;
44     int _day;
45 };
46
47 int main()
48 {
49     Date d1(2024, 7, 5);
50     Date d2(2024, 7, 6);
51
52     // 运算符重载函数可以显示调用
53     d1.operator==(d2);
54
55     // 编译器会转换成 d1.operator==(d2);
56     d1 == d2;
57
58     // 编译器会转换成 d1.operator++();
59     ++d1;
60     // 编译器会转换成 d1.operator++(0);
61     d1++;
62
63     return 0;
64 }

```

5.2 赋值运算符重载

赋值运算符重载是一个默认成员函数，用于完成两个已经存在的对象直接的拷贝赋值，这里要注意跟拷贝构造区分，拷贝构造用于一个对象拷贝初始化给另一个要创建的对象。

赋值运算符重载的特点：

1. 赋值运算符重载是一个运算符重载，规定必须重载为成员函数。赋值运算符重载的参数建议写成 `const` 当前类类型引用，否则会传值传参会有拷贝
2. 有返回值，且建议写成当前类类型引用，引用返回可以提高效率，有返回值目的是为了支持连续赋值场景。
3. 没有显式实现时，编译器会自动生成一个默认赋值运算符重载，默认赋值运算符重载行为跟默认拷贝构造函数类似，对内置类型成员变量会完成值拷贝/浅拷贝(一个字节一个字节的拷贝)，对自定义类型成员变量会调用他的赋值重载函数。

4. 像Date这样的类成员变量全是内置类型且没有指向什么资源，编译器自动生成的赋值运算符重载就可以完成需要的拷贝，所以不需要我们显示实现赋值运算符重载。像Stack这样的类，虽然也都是内置类型，但是_a指向了资源，编译器自动生成的赋值运算符重载完成的值拷贝/浅拷贝不符合我们的需求，所以需要我们自己实现深拷贝(对指向的资源也进行拷贝)。像MyQueue这样的类型内部主要是自定义类型Stack成员，编译器自动生成的赋值运算符重载会调用Stack的赋值运算符重载，也不需要我们显示实现MyQueue的赋值运算符重载。这里还有一个小技巧，如果一个类显示实现了析构并释放资源，那么他就需要显示写赋值运算符重载，否则就不需要。

```
1 class Date
2 {
3 public:
4     Date(int year = 1, int month = 1, int day = 1)
5     {
6         _year = year;
7         _month = month;
8         _day = day;
9     }
10
11     Date(const Date& d)
12     {
13         cout << " Date(const Date& d)" << endl;
14
15         _year = d._year;
16         _month = d._month;
17         _day = d._day;
18     }
19
20     // 传引用返回减少拷贝
21     // d1 = d2;
22     Date& operator=(const Date& d)
23     {
24         // 不要检查自己给自己赋值的情况
25         if (this != &d)
26         {
27             _year = d._year;
28             _month = d._month;
29             _day = d._day;
30         }
31
32         // d1 = d2表达式的返回对象应该为d1, 也就是*this
33         return *this;
34     }
35
36     void Print()
37     {
```

```

38         cout << _year << "-" << _month << "-" << _day << endl;
39     }
40 private:
41     int _year;
42     int _month;
43     int _day;
44 };
45
46
47 int main()
48 {
49     Date d1(2024, 7, 5);
50     Date d2(d1);
51
52     Date d3(2024, 7, 6);
53     d1 = d3;
54
55     // 需要注意这里是拷贝构造，不是赋值重载
56     // 请牢牢记住赋值重载完成两个已经存在的对象直接的拷贝赋值
57     // 而拷贝构造用于一个对象拷贝初始化给另一个要创建的对象
58     Date d4 = d1;
59
60     return 0;
61 }

```

5.3 日期类实现

```

1  #pragma once
2  #include<iostream>
3  using namespace std;
4
5  #include<assert.h>
6
7  class Date
8  {
9      // 友元函数声明
10     friend ostream& operator<<(ostream& out, const Date& d);
11     friend istream& operator>>(istream& in, Date& d);
12
13 public:
14     Date(int year = 1900, int month = 1, int day = 1);
15     void Print() const;
16
17     // 直接定义类里面，他默认是inline
18     // 频繁调用

```

```

19     int GetMonthDay(int year, int month)
20     {
21         assert(month > 0 && month < 13);
22
23         static int monthDayArray[13] = { -1, 31, 28, 31, 30, 31, 30,
31, 31, 30, 31, 30, 31 };
24
25         // 365天 5h +
26         if (month == 2 && (year % 4 == 0 && year % 100 != 0) || (year
% 400 == 0))
27         {
28             return 29;
29         }
30         else
31         {
32             return monthDayArray[month];
33         }
34     }
35
36     bool CheckDate();
37
38     bool operator<(const Date& d) const;
39     bool operator<=(const Date& d) const;
40     bool operator>(const Date& d) const;
41     bool operator>=(const Date& d) const;
42     bool operator==(const Date& d) const;
43     bool operator!=(const Date& d) const;
44
45     // d1 += 天数
46     Date& operator+=(int day);
47     Date operator+(int day) const;
48
49     // d1 -= 天数
50     Date& operator-=(int day);
51     Date operator-(int day) const;
52
53     // d1 - d2
54     int operator-(const Date& d) const;
55
56     // ++d1 -> d1.operator++()
57     Date& operator++();
58
59     // d1++ -> d1.operator++(0)
60     // 为了区分，构成重载，给后置++，强行增加了一个int形参
61     // 这里不需要写形参名，因为接收值是多少不重要，也不需要写
62     // 这个参数仅仅是为了跟前置++构成重载区分
63     Date operator++(int);

```

```
64
65     Date& operator--();
66     Date operator--(int);
67
68     // 流插入
69     // 不建议, 因为Date* this占据了一个参数位置, 使用d<<cout不符合习惯
70     //void operator<<(ostream& out);
71 private:
72     int _year;
73     int _month;
74     int _day;
75 };
76
77 // 重载
78 ostream& operator<<(ostream& out, const Date& d);
79 istream& operator>>(istream& in, Date& d);
80
81 // Date.cpp
82 #include "Date.h"
83
84 bool Date::CheckDate()
85 {
86     if (_month < 1 || _month > 12
87         || _day < 1 || _day > GetMonthDay(_year, _month))
88     {
89         return false;
90     }
91     else
92     {
93         return true;
94     }
95 }
96
97 Date::Date(int year, int month, int day)
98 {
99     _year = year;
100     _month = month;
101     _day = day;
102
103     if (!CheckDate())
104     {
105         cout << "日期非法" << endl;
106     }
107 }
108
109 void Date::Print() const
110 {
```

```

111         cout << _year << "-" << _month << "-" << _day << endl;
112     }
113
114     // d1 < d2
115     bool Date::operator<(const Date& d) const
116     {
117         if (_year < d._year)
118         {
119             return true;
120         }
121         else if (_year == d._year)
122         {
123             if (_month < d._month)
124             {
125                 return true;
126             }
127             else if (_month == d._month)
128             {
129                 return _day < d._day;
130             }
131         }
132
133         return false;
134     }
135
136     // d1 <= d2
137     bool Date::operator<=(const Date& d) const
138     {
139         return *this < d || *this == d;
140     }
141
142     bool Date::operator>(const Date& d) const
143     {
144         return !(*this <= d);
145     }
146
147     bool Date::operator>=(const Date& d) const
148     {
149         return !(*this < d);
150     }
151
152     bool Date::operator==(const Date& d) const
153     {
154         return _year == d._year
155             && _month == d._month
156             && _day == d._day;
157     }

```

```

158
159 bool Date::operator!=(const Date& d) const
160 {
161     return !(*this == d);
162 }
163
164 // d1 += 50
165 // d1 += -50
166 Date& Date::operator+=(int day)
167 {
168     if (day < 0)
169     {
170         return *this -= -day;
171     }
172
173     _day += day;
174     while (_day > GetMonthDay(_year, _month))
175     {
176         _day -= GetMonthDay(_year, _month);
177         ++_month;
178         if (_month == 13)
179         {
180             ++_year;
181             _month = 1;
182         }
183     }
184
185     return *this;
186 }
187
188
189 Date Date::operator+(int day) const
190 {
191     Date tmp = *this;
192     tmp += day;
193
194     return tmp;
195 }
196
197 // d1 -= 100
198 Date& Date::operator-=(int day)
199 {
200     if (day < 0)
201     {
202         return *this += -day;
203     }
204

```

```

205     _day -= day;
206     while (_day <= 0)
207     {
208         --_month;
209         if (_month == 0)
210         {
211             _month = 12;
212             _year--;
213         }
214
215         // 借上一个月的天数
216         _day += GetMonthDay(_year, _month);
217     }
218
219     return *this;
220 }
221
222 Date Date::operator-(int day) const
223 {
224     Date tmp = *this;
225     tmp -= day;
226
227     return tmp;
228 }
229
230 //++d1
231 Date& Date::operator++()
232 {
233     *this += 1;
234
235     return *this;
236 }
237
238 // d1++
239 Date Date::operator++(int)
240 {
241     Date tmp(*this);
242     *this += 1;
243
244     return tmp;
245 }
246
247 Date& Date::operator--()
248 {
249     *this -= 1;
250     return *this;
251 }

```

```

252
253 Date Date::operator--(int)
254 {
255     Date tmp = *this;
256     *this -= 1;
257
258     return tmp;
259 }
260
261 // d1 - d2
262 int Date::operator-(const Date& d) const
263 {
264     Date max = *this;
265     Date min = d;
266     int flag = 1;
267     if (*this < d)
268     {
269         max = d;
270         min = *this;
271         flag = -1;
272     }
273
274     int n = 0;
275     while (min != max)
276     {
277         ++min;
278         ++n;
279     }
280
281     return n * flag;
282 }
283
284 ostream& operator<<(ostream& out, const Date& d)
285 {
286     out << d._year << "年" << d._month << "月" << d._day << "日" << endl;
287     return out;
288 }
289
290 istream& operator>>(istream& in, Date& d)
291 {
292     cout << "请依次输入年月日:>";
293     in >> d._year >> d._month >> d._day;
294
295     if (!d.CheckDate())
296     {
297         cout << "日期非法" << endl;
298     }

```

```
299
300         return in;
301     }
302
303     // Test.cpp
304     #include "Date.h"
305     void TestDate1()
306     {
307         // 这里需要测试一下大的数据+和-
308         Date d1(2024, 4, 14);
309         Date d2 = d1 + 30000;
310         d1.Print();
311         d2.Print();
312
313         Date d3(2024, 4, 14);
314         Date d4 = d3 - 5000;
315         d3.Print();
316         d4.Print();
317
318         Date d5(2024, 4, 14);
319         d5 += -5000;
320         d5.Print();
321     }
322
323     void TestDate2()
324     {
325         Date d1(2024, 4, 14);
326         Date d2 = ++d1;
327         d1.Print();
328         d2.Print();
329
330         Date d3 = d1++;
331         d1.Print();
332         d3.Print();
333
334         /*d1.operator++(1);
335         d1.operator++(100);
336         d1.operator++(0);
337         d1.Print();*/
338     }
339
340     void TestDate3()
341     {
342         Date d1(2024, 4, 14);
343         Date d2(2034, 4, 14);
344
345         int n = d1 - d2;
```

```

346         cout << n << endl;
347
348         n = d2 - d1;
349     }
350
351     void TestDate4()
352     {
353         Date d1(2024, 4, 14);
354         Date d2 = d1 + 30000;
355
356         // operator<<(cout, d1)
357         cout << d1;
358         cout << d2;
359
360         cin >> d1 >> d2;
361         cout << d1 << d2;
362     }
363
364     void TestDate5()
365     {
366         const Date d1(2024, 4, 14);
367         d1.Print();
368
369         //d1 += 100;
370         d1 + 100;
371
372         Date d2(2024, 4, 25);
373         d2.Print();
374
375         d2 += 100;
376
377         d1 < d2;
378         d2 < d1;
379     }
380
381     int main()
382     {
383         return 0;
384     }

```

6. 取地址运算符重载

6.1 const成员函数

- 将const修饰的成员函数称之为const成员函数，const修饰成员函数放到成员函数参数列表的后面。

- `const`实际修饰该成员函数隐含的`this`指针，表明在该成员函数中不能对类的任何成员进行修改。
`const` 修饰`Date`类的`Print`成员函数，`Print`隐含的`this`指针由 `Date* const this` 变为 `const Date* const this`

```
1 #include<iostream>
2 using namespace std;
3
4 class Date
5 {
6 public:
7     Date(int year = 1, int month = 1, int day = 1)
8     {
9         _year = year;
10        _month = month;
11        _day = day;
12    }
13
14    // void Print(const Date* const this) const
15    void Print() const
16    {
17        cout << _year << "-" << _month << "-" << _day << endl;
18    }
19 private:
20     int _year;
21     int _month;
22     int _day;
23 };
24
25 int main()
26 {
27     // 这里非const对象也可以调用const成员函数是一种权限的缩小
28     Date d1(2024, 7, 5);
29     d1.Print();
30
31     const Date d2(2024, 8, 5);
32     d2.Print();
33
34     return 0;
35 }
```

6.2 取地址运算符重载

取地址运算符重载分为普通取地址运算符重载和`const`取地址运算符重载，一般这两个函数编译器自动生成的就可以够我们用了，不需要去显示实现。除非一些很特殊的场景，比如我们不想让别人取到当

前类对象的地址，就可以自己实现一份，胡乱返回一个地址。

```
1 class Date
2 {
3 public :
4     Date* operator&()
5     {
6         return this;
7         // return nullptr;
8     }
9
10    const Date* operator&()const
11    {
12        return this;
13        // return nullptr;
14    }
15 private :
16    int _year ; // 年
17    int _month ; // 月
18    int _day ; // 日
19 };
```